

THE MEASURING EQUIPMENT

Bitcoin Machine Code, days 22 to 38: the state of the project on
2026-09-19

A follow-up to "21 FOR 21"

Bitcoin Machine Code · github.com/BobClawblaw/bitcoinmachinecode

A follow-up to "21 FOR 21", which covered 2026-08-11 to 2026-09-02. This report covers 2026-09-02 to 2026-09-19. It was compiled from the project's own records: the git history of <https://github.com/BobClawblaw/bitcoinmachinecode>, the pull requests, the daily worklogs, docs/devlog (LOG.md, DEPLOYMENT_HISTORY.md, the RESUME and incident notes), docs/reports (the IBD reports), the registers (CORE_DIVERGENCES, CORE_BEHAVIORAL_COMPAT, FEATURE_GAPS, PARITY_RPC_FIELDS), the audits, the benchmark harness logs on disk, and the assistant's own persistent memory notes. Every quote is verbatim from one of those. As in the first report, the counts are true as of the date given and will drift.

```
"The measuring equipment was not trustworthy."  
-- worklog/2026-09-13.md
```

Contents

PROLOGUE: WHERE THE FIRST REPORT STOPPED.....	5
PART I: THE NUMBERS – what seventeen days added.....	6
Chapter 1: Counting commits when the history exists twice	6
Chapter 2: Lines of code	7
Chapter 3: Who wrote it	7
PART II: WHAT WAS BUILT – from a node to a node that behaves like Core.....	8
Chapter 4: The audit, and the start of the pull-request era (09-03 to 09-05)	8
Chapter 5: The UTX0 set moves inside the download (09-06)	8
Chapter 6: The download in Core's shape (09-07 to 09-10)	8
Chapter 7: The node as a backend (09-08)	9
Chapter 8: Every index during the sync (09-16)	9
Chapter 9: The mempool (09-14 to 09-16)	9
Chapter 10: The archive frontier (09-17, 09-18)	10
Chapter 11: The last three days (09-17 to 09-19)	10
PART III: THE LEDGER – consensus, and one block the AI destroyed.....	11
Chapter 12: Nine comparisons with Core	11
Chapter 13: "If core has no cap, we should not have one either" (09-09)	11
Chapter 14: Two more ways to be wrong about coins	12
Chapter 15: Block 967,422	12
PART IV: THE MEASURING EQUIPMENT – seventeen days of benchmarks that lied.....	14
Chapter 16: The capstone that never ran	14
Chapter 17: The Core baseline was handicapped by us	15
Chapter 18: Two disks, not two implementations	15
Chapter 19: Run 27, and what was wrong with it	16
Chapter 20: The clean pair	17
Chapter 21: Where the time goes	19
PART V: PARITY – a name is not a contract.....	20
Chapter 22: Method names, and then shapes	20
Chapter 23: The wrong Core	20
Chapter 24: ZMQ	21
Chapter 25: Relay	22
Chapter 26: What still differs, on purpose	22
PART VI: SECURITY – 182 findings, one that fell through, and a leak.....	23
Chapter 27: Closing an audit by ID	23
Chapter 28: Keys in a backup folder	24
PART VII: THE SCAR TISSUE – incidents, and the patterns they share.....	25
Chapter 29: A green suite that measured nothing	25
Chapter 30: DONE on paper, zero at runtime	25
Chapter 31: A comment that ends the inquiry	26
Chapter 32: A threshold calibrated at one point	26
Chapter 33: The tools as an adversary	26

Chapter 34: Misreading the human	27
Chapter 35: The rules now in force	27
PART VIII: THE COLLABORATION – from standing authorization to a gated contract	29
Chapter 36: The operator as the merge and deploy button	29
Chapter 37: Parallel agents	29
Chapter 38: Memory as the institutional record	30
Chapter 39: Neighbours	30
PART IX: WHAT IS STILL OPEN	31
APPENDIX A: THE NUMBERS, ON ONE PAGE	32
APPENDIX B: RECOVERING A TIMELINE FROM BLOCK FILES	33
APPENDIX C: READING ORDER – AUDITING THIS REPORT AGAINST THE SOURCE	34

PROLOGUE: WHERE THE FIRST REPORT STOPPED

"21 FOR 21" ended on 2026-09-02 with a node, written entirely in hand-crafted x86-64 assembly and C with no human-typed code, whose UTX0 set was MuHash-identical to Bitcoin Core's at a chain height of 963,967. It ended on a question it could not yet answer: was the node any good at the thing a node is for, syncing the chain from nothing and keeping up with it, measured honestly against Core on the same machine?

Seventeen days later the answer is "probably, and here is exactly how sure we are". Most of this report is about that last clause.

The short version of the state on 2026-09-19:

- Correctness. A fresh bmc sync (run 27) ended with a UTX0 set whose MuHash is identical to Bitcoin Core v31.1's at height 967,712: 09f5cd877189dc269f505f2066aa2a0b39eb0b8e3384a48449880affd6490ee7. That is the ninth full-set comparison against Core in the period. Every one that ran matched. The two that "failed" were both failures of the measuring harness, not of the node (Part III).
- Speed. On the same NVMe drive, run 27 reached the tip in 18 h 40 m 23 s. The first Core v31.1 baseline took 19 h 40 m 09 s. That is about 60 minutes (~5%) in bmc's favour. The report that records it says "Treat that as indicative, not clean", and so does this one: both runs were loaded by a monitoring tool, in different ways (Part IV). The clean pair, an unpolled Core rerun and run 28 on today's code, was in progress when this was written. Both finished by 2026-09-21: Core v31.1 left IBD in 19 h 32 m 54 s and run 28 in 18 h 24 m 02 s. That is 1 h 08 m 52 s (5.9%) in bmc's favour, and run 28's UTX0 set was MuHash-identical to Core's at 968,025 (Chapter 20).
- Parity. Every one of Core v31.1's RPC method names exists on the node and none is a stub. More usefully, the response shapes are now diffed field by field against a real v31.1, which is what found most of the defects in this period (Part V).
- The work. 927 commits on main and 282 merged pull requests in seventeen days, written by three Claude models, with one human operator deciding, merging and deploying (Parts I and VIII).

The title is not a metaphor. In this period, the most expensive defects were not in the node. They were in the things that measured it: a capstone that had compared an empty string with an empty string "for months", a harness that read a moving UTX0 set and reported corruption, a benchmark that compared two disks instead of two implementations, a parity check against the wrong version of Core, a monitor that loaded both sides of a timed race. Each was found, named and fixed, and each left a rule behind. That is the main result of these seventeen days, and it is what makes the other results believable.

PART I: THE NUMBERS – what seventeen days added

Chapter 1: Counting commits when the history exists twice

A warning first, because it changes every commit count in this report. On 2026-09-04 two rewritten copies of the project history were reconciled into one main branch:

```
"origin/main and this branch are two REWRITES of the same project history
-- they share only an ancient base, 881 commits on one side and 956 on the
other"
-- commit dfbdf36d, 2026-09-04
```

The merge kept one tree and recorded the other side as a second parent, so that nothing was force-overwritten. As a result the same logical commit often appears twice in `git log`, and a raw `git rev-list --count main` (2,844) roughly doubles the older work. The honest figure excludes the discarded side (`main ^7de98563`):

	2026-09-02	2026-09-19
commits on main (deduplicated)	1,036	1,774
commits on main since 09-02		927 (642 non-merge, 285 merges)
merged pull requests	0 (work went straight to main)	282 (#2 to #283)
test programs in the gate	324	428
RPC methods in the dispatch tables	161	166
tags	3	108

"21 FOR 21" quoted 1,023 commits on main for 2026-09-02. That matches the count at about 14:00Z that day; the last commit of the day brings it to 1,036.

The pull-request era began on 2026-09-05. Before that, work was committed straight to main. After a forum reviewer asked for squashed history, the operator chose the opposite: every batch of work lands as one pull request and one merge commit, never squashed, never force-pushed, so that `git log --first-parent main` reads as one line per batch while the granular commits stay intact. The busiest day was 2026-09-06, with 48 merged pull requests. On 2026-09-11 there were none.

Chapter 2: Lines of code

	2026-09-02	2026-09-19	change
assembly, source	57,001	60,477	+3,476
assembly, tests	11,187	11,187	0
C, source	71,797	97,872	+26,075
C, tests	87,288	116,837	+29,549
Python	22,230	24,415	+2,185
shell	4,223	7,420	+3,197
total, code and tests	~253,700	~318,200	+25%

Raw lines, comments and blank lines included; tests are paths under tests/ or files named test_*. The book's own "~166,000 lines" used a different method, so the two figures are not comparable.

The shape of the growth matters more than its size. The assembly barely moved. Everything that was added (the parallel download, the indexes, the RPC and REST surfaces, the Esplora facade, the cluster mempool, the ZMQ publisher) was added in C around an assembly core that stayed stable: SHA-256, the script interpreter, the secp256k1 arithmetic, the block and transaction parsers, the archive store, the UTXO engine's inner loops. The tests grew faster than the code: 29,549 new lines of C tests against 26,075 new lines of C source.

Chapter 3: Who wrote it

Every commit is authored as the project's pseudonymous account. The models that wrote them are named in Co-Authored-By trailers:

model	commits co-authored since 09-02 (main, deduplicated)
Claude Fable 5.1	310
Claude Opus 5 (1M context)	269
Claude Sonnet 5	10

The hand-offs are visible by date. Sonnet 5 appears only on 09-03. Opus 5 carried the audit remediation on 09-04 and 09-05. Fable 5.1 carried 09-06 to 09-10 almost alone. From 09-11, it is Opus 5. "21 FOR 21" named seven models, several of them outside Claude; in this period, no model outside Claude appears in a trailer. The 09-03 codebase audit itself was run by Claude Fable 5.1 as thirteen parallel module reviewers.

62 of the 642 non-merge commits carry no trailer. The commits trace to five separate Claude sessions, linked by hand-off files (Part VIII).

PART II: WHAT WAS BUILT – from a node to a node that behaves like Core

The first report described a node that could validate the chain. The work of these seventeen days was mostly about behaving like Core while doing it: the same download shape, the same chain selection, the same RPC shapes, the same indexes, the same notifications. The operator's instruction, repeated in different words on several days, was the same: "Do what core does!"

Chapter 4: The audit, and the start of the pull-request era (09-03 to 09-05)

On 2026-09-03 a full module-by-module codebase audit ran as thirteen parallel reviewers. It returned 182 findings: 5 CRITICAL, 32 HIGH, 44 MEDIUM, 68 LOW and 33 INFO. Its headline:

```
"a block Core rejects can be accepted, stored and applied by this node"  
-- docs/audits/CODEBASE_AUDIT_2026-09-03.md
```

The coinbase-value check (VAL-1) and the MAX_MONEY check (VAL-2) were among them; the only MAX_MONEY check "lives in a function with zero call sites". Three interpreter findings were false accepts, and one (SCR-2) was an out-of-bounds write an attacker could reach. All 29 distinct CRITICAL and HIGH findings were closed by 14:54Z on 09-04. Part VI covers how closure was tracked, and the one finding that fell through the tracking.

Chapter 5: The UTXO set moves inside the download (09-06)

On 09-04 a fresh install synced in 24.1 hours: 19.5 hours of download at 11.1 MB/s, then 4.5 hours of a serial UTXO catch-up. The gap to Core was not a slower UTXO engine; it was a UTXO engine that ran after the download instead of during it. On 09-06 the connect moved inside the parallel download (#27, tag `utxo-interleave-2026-09-06`). The same day's decomposition of the remaining gap found the largest single term: a MuHash fold per coin on the bulk path, about 1.66 μ s times some 6.4 billion coin events, roughly three hours. The 3072-bit modular multiply went from 944 ns to 301 ns with BMI2, ADX and AVX-512 IFMA; the flush sort went from a 541 ms merge sort to a 91 ms radix sort for four million keys.

Chapter 6: The download in Core's shape (09-07 to 09-10)

Over four days the parallel downloader was rebuilt to match Core's design (#77, #78, #157): every live peer downloads, blocks are requested in 40-block chunks within a window anchored to the connected tip, and only the tail of that window

judges whether a peer is too slow. One in-order committer appends the archive in height order (#97). Download concurrency defaults to eight peers, because Core's is pinned at eight: sixteen workers against sixty-four had measured 11.2 against 11.3 MB/s, "Four times the workers buys about one percent" (#178).

Chain selection became Core's on 09-09 (#137): a fork tree that retains both branches' headers, after a peer stuck on a real stale branch held a benchmark for four hours. Compact blocks, marked DONE on 09-06, were finally received on 09-09 (#145; Chapter 30 explains the three-day gap). BIP324 v2 sessions were handed across a fork to dial helpers on 09-10 (#167, #174).

Chapter 7: The node as a backend (09-08)

On 09-08 the node started serving mempool.space in process through an Esplora HTTP facade (#100), gained an address history index (#107, #108), Core's REST interface (#115, checked byte for byte against Core with 82 route-level checks), and a coinstats history that answers `gettxoutsetinfo` at any height (#119, #120). Undo data is kept for every block, as Core's rev files are (#99).

Chapter 8: Every index during the sync (09-16)

The operator's instruction on 09-16:

"Make it so there is nothing left to do after ibd but just sit and update. Everything should be generated during ibd."

Since #238, all five indexes (txindex, txospenderindex, coinstatsindex, blockfilterindex and the address index) are built while the chain syncs. Run 26 was the first run to finish with all five complete.

Chapter 9: The mempool (09-14 to 09-16)

The cluster mempool landed in five stages on 09-14 (#197 to #203): linearization, cluster discovery, `getmempoolcluster` on real clusters, post-linearization, one bound. The plan was corrected on the way: "two of its seven stages were already done". Block templates and eviction were moved onto the same tested linearization (#204 to #207).

On 09-16 the node gained something Core does not have: a departure journal (#220 onward). Core forgets an evicted or expired transaction entirely, so "what happened to the one I broadcast" has no answer. The journal records one 152-byte entry per departure, naming why the transaction left: mined, replaced, evicted, expired or conflicted. It is sized for about eleven days of mainnet traffic.

Chapter 10: The archive frontier (09-17, 09-18)

The block archive's append cursor could fall behind the newest block file after a restart, and then silently fill the leftover tail of every older file on its way forward, putting a lower offset at a higher height. Run 26 had six such "layout breaks", each the first block after a restart. The fix took four pull requests and, on the way, destroyed a block (Chapter 15).

Chapter 11: The last three days (09-17 to 09-19)

The final days were a parity push against a real Core v31.1 (Part V), fixes to the ZMQ publisher so that notifications are delivered at all (Chapter 24), transaction relay defects found by a regtest differential (Chapter 25), a restart that waits for every process holding the datadir lock, and the changes that make run 28 a fair measurement: every index gated on its config key as in Core, every helper binary built, and no RPC to the node while it syncs.

PART III: THE LEDGER – consensus, and one block the AI destroyed

Chapter 12: Nine comparisons with Core

A UTX0 set comparison is the strongest correctness check this project has: the node's entire set of unspent coins, hashed with MuHash, against Core's at the same height. Every one run in the period:

date	node	height	result
09-02	production	965,135	identical, 165,632,732 coins
09-04	fresh-install script	–	printed PASS on two empty strings
09-05	two independently synced datadirs	965,651	identical
09-09	production, rebuilt coinstats history	963,967	identical in every field
09-11	run 22, live	966,494	FAIL: a torn read, not a divergence
09-12	run 22, walked offline	966,496	identical on all five fields
09-12	run 23	966,674	identical
09-17	run 26, five indexes built during the sync	967,450 and 200,000	identical at both
09-19	run 27, same NVMe as the Core baseline	967,712	identical

Sources: docs/PARITY_ATTESTATION.md, docs/reports/*, the run 27 phase.log.

Two rows are not results. On 09-04 the fresh-install script compared two empty strings, found them equal and printed PASS. On 09-11 the capstone read the live node's set while its UTX0 engine was still applying blocks and reported a failure. Both are harness defects, covered in Part IV. Every comparison that actually compared two sets matched.

Chapter 13: "If core has no cap, we should not have one either" (09-09)

At 00:31Z on 09-09 a benchmark, 91% through a fresh sync, rejected valid mainnet block 880,338 as `bad-txns-nonBIP68-final`: one transaction had 7,244 inputs, and the node recorded at most 2,048 sequence numbers per transaction. That cap had been a deliberate choice on 09-04 (VAL-4). Production carried the same code.

```
"A valid block, rejected; a consensus failure. Production ... would have forked on the next such block."
```

```
-- docs/releases/2026-09-09-bip68-seq-cap.md
```

The first fix derived a new cap from the block weight limit: 24,576. The operator's answer was short:

```
"If core has no cap, we should not have one either. Follow their behavior wherever possible for consensus."
```

The cap was removed the same night (#129). The rule now reads: "Consensus follows Core's behaviour wherever Core has one; where Core has none, we invent none." A follow-up audit (#131) found two more places where the node bounds something Core does not, and both are still open: a UTXO record stores a script's length in two bytes, so a valid output script over 65,535 bytes cannot be stored, and `submitblock` refuses blocks with more than 16,384 transactions where a valid block could hold 16,666.

Chapter 14: Two more ways to be wrong about coins

On 09-09 the coinstats history disagreed with Core from height 91,722. Core's `IsBIP30Unspendable` names the two original coinbases that were later duplicated; the node named the duplicates instead. The coin counts converged, but every affected coin hashed with the wrong height (#133).

The same day, run 18 rejected mainnet block 963,311 for spending a coin that was present. One compaction run held 11,362 keys twice. The `live=` figure in the logs agreed with Core's count throughout, because "The `live=` figure in the logs is a counter, not a scan, so an equal count proved nothing." One of the existing tests had expected the duplicate tombstones: a test that encoded the defect (#139).

Chapter 15: Block 967,422

On 2026-09-17, fixing the archive append frontier (Chapter 10), the assistant wrote a guard on the C side that advanced the cursor's file number without its position. The next append wrote 293 bytes of the genesis block over offset 0 of `blk05762.dat`, where block 967,422 began, and the node then served a one-transaction block under that block's correct hash. The hand-off file for the next session opens:

```
"Block 967422 on run 26 is corrupt and I caused it."
```

```
-- docs/devlog/RESUME_2026-09-17.md
```

The repair was a single 293-byte write, reconstructed from production's copy of the same block. The permission classifier that governs the assistant's actions blocked it, "correctly, it guards writes into archive files", until the

operator gave an explicit go-ahead. The work was committed as "WIP (NOT REVIEWED)" and left unmerged on purpose.

The next day the block was repaired and hash-verified, and a closer look found the damage was worse than the write offset: the bytes had gone to the old file while the index named the new one. Three further pull requests (#266 to #268) made the store record which file a position was measured in, and the guard was then proven on a live restart that put the cursor 2,363 files behind the frontier. The next block landed exactly at the end of the frontier file. Six layout breaks before the restart, six after.

The damage was confined to the stored copy of that one block: "Everything else in the archive is intact; the rest of that block is byte-identical to Core's copy" (RESUME_2026-09-17.md). The six older layout breaks never affected the chain either, though they still make truncation and pruning refuse to run on run 26's archive.

PART IV: THE MEASURING EQUIPMENT – seventeen days of benchmarks that lied

The central question after "21 FOR 21" was how long a fresh sync takes against Core on the same machine. Twenty-seven numbered runs later, the project has one comparison it is willing to call indicative, and a second one in flight that it means to call clean. What took seventeen days was not making the node faster. It was making the measurement true.

Chapter 16: The capstone that never ran

Every benchmark run ends with a capstone: hash the node's UTXO set, hash Core's at the same height, compare. On 2026-09-11, run 22's capstone failed, and the report written that day contains this sentence:

```
"This is the first time the muhash comparison has ever completed. Earlier runs
timed out, and an empty result compared equal to an empty oracle value, so the
harness printed nothing and the run passed."
-- docs/reports/2026-09-11-run22-muhash-divergence.md
```

Every "passing" run before 2026-09-11 had passed a check that never executed. The fix was a rule the harness still carries: an empty hash is a failure, and a value that is not 64 hex characters from both sides is refused.

The failure it then reported was also false. The capstone had hashed the live node's set six minutes after the tip, while the UTXO engine was still applying blocks it had already stored. `setnetworkactive false` stopped new blocks arriving but not the engine. The report concluded that fresh syncs write correct coins with wrong height metadata, because every aggregate matched and only the hash differed. The next day, the stored set was walked offline, quiesced, and matched Core on all five fields. The correction reads:

```
"A walk over a moving LSM is not a set."
"Aggregates agreeing with a hash that disagrees is the signature of a torn read."
```

The aggregates agreed because `txouts` on the live path is a maintained counter, not the walk's own count. The capstone now quiesces the node, asks it for its current set with no height argument, and pins Core to whatever height that answer reports.

Run 23 then produced no hash at all: the harness had asked its own node for the set at a height its `coinstats` index had not reached yet. The Core baseline on 09-13 produced an empty hash three times, because the call did not name a hash type and Core's default response has no `muhash` key. The new guard "did its job: it reported FAIL with both values printed rather than comparing empty to empty." A gate check now requires every such call to name `muhash`.

A census of the harness on 09-12 found "FIVE defects in the benchmark harness and ZERO in the node". The progress heartbeat had recorded nothing for three runs, because it read the console log instead of debug.log, matched `[dl]` against lines tagged `[dlc]`, and ran grep without `-a` on a log containing NUL bytes, which makes grep print nothing at all. 09-13 was spent almost entirely on instruments. All seventeen differential scripts were found to print "ALL 0 MATCH" and exit 0 when their case set was empty; they now exit 2.

Chapter 17: The Core baseline was handicapped by us

On 09-06 the Core benchmark script turned out to set `par=8` and `maxconnections=64` for Core, "added 2026-09-04; no justification in the script, the memo or the commit". The 21-hour Core figure every comparison had used was measured with Core on eight verification threads and half its default connections. Separately, bmc's own `par` key set its number of download workers, not verification threads. The operator's response became two rules:

```
"Our flags need to match core. Never do something different unless we discuss it."
"Don't ever take liberties with Core again."
```

Three more asymmetries surfaced over the next week. Every published comparison had run bmc with 16 to 64 downloading peers against Core's fixed 8 (09-10). The Core runner had used `ionice -c3`, the idle I/O class, for Core only (09-12). And the "hour ahead of Core" in the 09-11 report came down to one bad peer: Core's run had spent 2 h 18 m on heights 835,000 to 840,000, a stretch every other run crossed in 11 to 16 minutes. The corrected report says the published version was "wrong in both directions", and concludes:

```
"on this box, at 8 peers a side, the two implementations are within a few
percent ... A single number from any one run is noise."
-- docs/reports/2026-09-11-ibd-vs-core.md, correction
```

Chapter 18: Two disks, not two implementations

On 09-17 run 26 finished in 19 h 34 m, having built all five indexes during the sync. Its report set that beside a clean Core baseline of 19 h 14 m. The comparison was withdrawn within hours:

```
"A Core run handicapped by a portable SSD against a bmc run on NVMe is a
comparison of disks, not of implementations."
"There is currently no valid IBD wall-clock comparison between this node and
Core."
-- docs/reports/2026-09-17-run26-vs-core.md, correction
```

Core had run on a Samsung Portable SSD T5 (0.40 GiB/s) and run 26 on an NVMe. The report's original caveats (ten boots, a binary swap mid-run, Core's unrecorded index settings) had "missed the one that mattered most".

That night an 8 TB NVMe was installed, and a protocol written down: the same device for both, a fresh datadir, `txindex`, `coinstatsindex` and `blockfilterindex`, `dbcache=8192`, eight download peers, no `ionice`, and each side's config copied into the report before launch. The Core version was fixed at v31.1, the release the node targets and every earlier baseline's version, not the v31.99 development build that serves as the box's oracle. A first Core attempt with debug logging on was stopped after ten minutes and moved aside, so that its logging cost would not count against Core.

Chapter 19: Run 27, and what was wrong with it

The Core v31.1 baseline ran from 2026-09-17 23:26:54Z until it logged `Leaving InitialBlockDownload` at 2026-09-18 19:07:03Z, at height 967,454: 19 h 40 m 09 s. Its process was restarted once, for twelve seconds at 06:12Z, to move it under systemd, and resumed with a cold cache.

Run 27 started on the same NVMe at 19:15:29Z on 09-18, on build `ad77e46e`. It had every block stored after 18 h 36 m 24 s, applied the tip 18 h 40 m 23 s in (13:55:52Z in its log), and its harness, which checks every five minutes, recorded the tip at 18 h 43 m 11 s. Its capstone:

```
2026-09-19T13:59:41Z PASS muhash identical at 967712
(09f5cd877189dc269f505f2066aa2a0b39eb0b8e3384a48449880affd6490ee7)
```

At every 50,000-block mark from 400,000 on, run 27 was ahead of Core's time for the same height. Core's early timeline had to be recovered from the creation times of its block files, because the twelve-second restart had trimmed its log (Appendix B describes the method).

height	Core v31.1	bmc run 27 (at most)	bmc ahead (at least)
400,000	1 h 40 m	1 h 35 m	5 m
500,000	3 h 50 m	3 h 41 m	9 m
600,000	6 h 14 m	5 h 56 m	18 m
700,000	9 h 08 m	8 h 48 m	20 m
800,000	12 h 24 m	12 h 02 m	22 m
900,000	16 h 51 m	16 h 13 m	38 m
950,000	18 h 58 m	18 h 03 m	55 m
tip	19 h 40 m 09 s	18 h 40 m 23 s	~60 m

The run's own report treats this as indicative, not clean, for these reasons:

1. Both runs were polled by a monitoring tool (Blockyard, a sibling project on the same machine), which the operator had asked to follow them. Core was polled for about 17 of its 19 h 40 m. Core's `gettxoutsetinfo` forces a UTXO cache flush on every call, so Core flushed its cache about once a minute for most of its run. Run 27 was polled for about 14 hours. On its

build, each poll re-read a large index tail from the start and re-walked the chain. Measured from the kernel's per-process I/O counters, run 27's RPC side read 10.2 TB over the run (203 MB/s on average), while the sync itself read 136 MB/s. In the minute after polling stopped, the RPC side read nothing.

2. Run 27 maintained an index nobody had asked for. The node started the txo-spender index tail on every boot, whatever the config said, so run 27 wrote an index Core did not (about 98 GB by the end).
3. Run 27 skipped work Core did. The harness had not built the helper binaries that fold the txindex into its final form, so the node logged that the builder was missing 222 times and never built it.
4. Run 27's final hour overlapped a test gate and a production restart on the same machine.

The first two burdened bmc, the third favoured it, and the polling burdened both sides differently. Most of bmc's lead was gained after 850,000, which is after its polling stopped at 09:07Z on 09-19, while Core's polling had continued to the end. The run's contaminants are listed in its report so that nobody quotes the 60 minutes without them.

Chapter 20: The clean pair

Every one of those contaminants now has a fix:

- Indexes run only when their config key is set, as in Core (#275).
- The harness builds every helper binary and refuses to start without them (#275).
- Trivial RPC calls answer in milliseconds during a sync (#280). Before, one slow call could hold the lock every other call waited on, and on run 27 `uptime` took 15 seconds.
- The harness makes no RPC call to the node until the download's own heartbeat says the sync is complete. Every five minutes it names any process connected to the node's RPC port and counts connections that closed in the last minute, which is how a poller that is in and out in milliseconds shows up (#281).

And an operator rule, from 2026-09-19:

```
"never add an IBD node to blockyard. Let the run finish before anything queries its RPC."
```

The Core rerun started at 14:28:33Z on 09-19, on a fresh datadir on the same NVMe, under its own systemd unit from its first second, not enabled and not restarted on failure, so that a crash or reboot would end the run visibly instead of resuming it quietly. It is watched without RPC: a script reads its log for `Leaving InitialBlockDownload` and the kernel's socket table for anyone calling in. Its first launch failed in its first second, and that was the

assistant's error: with no `bind=` line, Core also binds its onion listener on 127.0.0.1 at the P2P port plus one, and that was where the assistant had put RPC. A regtest agent had reported the same trap earlier that day, and the finding did not reach the benchmark config. The one-second-old datadir was moved aside, not deleted, and the rerun began 29 minutes after run 27 ended.

Results of the clean pair (filled in on 2026-09-21 from the runs' own logs):

	Core v31.1 rerun	bmc run 28
started	2026-09-19 14:28:33Z	2026-09-20 21:36:53Z
build	v31.1 9be056a8	main 1562ae86
left IBD / reached tip	2026-09-20 10:01:27Z, Leaving InitialBlockDownload at 967,690	2026-09-21 16:00:55Z, [dlc] catch-up done with the connected tip at 967,898
wall clock	19 h 32 m 54 s	18 h 24 m 02 s
UTX0 set vs Core	(the oracle)	MuHash identical at 968,025
strangers on the RPC port	none (one false positive, fixed in #285)	one burst: 15 connections closed within a minute at 05:07Z, caller not identified

bmc was 1 h 08 m 52 s ahead, 5.9% of Core's time. From 500,000 on, run 28 was 4% to 7% faster than Core in every 100,000-block segment. The two runs were sequential, on different days and different peers, and each side ran once.

Run 28 needed a harness repair before its capstone could run. Once the node logged `catch-up done`, it stopped writing the progress lines the harness read, so the monitor never saw the end and reported that the heartbeat had stopped moving. The monitor was restarted with the fix (PR #289); the node was not. The capstone ran at 17:45Z:

```
2026-09-21T17:45:46Z PASS muhash identical at 968025
(6e867a52c419fef00cec1768da7e8fbdf38a610c410cd4c4d01f02eef017fe2e)
```

The end time comes from the node's own log line, so the delay did not change it. The full comparison is in the separate report "Run 28 vs Core" ([docs/history/2026-09-21-run-28-vs-core/](#)).

This chapter expected the two Core runs to measure what the polling cost Core. They do not. From height 580,000, where the first run's log survives, the two runs differ by only 7 m 15 s at the end of IBD (0.6%), and the polled run was the faster one until somewhere between 850,000 and 900,000:

height	Core, polled (09-17/18)	Core, unpolled rerun (09-19/20)	polled minus unpolled, per segment
580,000	5 h 41 m 55 s	5 h 48 m 10 s	
650,000	7 h 36 m 46 s	7 h 47 m 13 s	-4 m 12 s
750,000	10 h 33 m 06 s	10 h 47 m 31 s	-3 m 58 s
850,000	14 h 37 m 50 s	14 h 45 m 53 s	+6 m 22 s
950,000	18 h 58 m 39 s	18 h 47 m 43 s	+18 m 59 s
left IBD	19 h 40 m 09 s	19 h 32 m 54 s	-3 m 41 s

Per 50,000-block segment the two runs differ by up to 8% in both directions. The first run also resumed from a cold cache after its twelve-second restart at 06:12Z, and the two runs left IBD at different heights (967,454 and 967,690). A difference that size cannot be attributed to the polling. What the pair does give is the first measure of how far two Core runs on the same machine drift apart: a few percent per segment, and under 1% over the whole sync.

Chapter 21: Where the time goes

- The download is flat at about 11 MB/s in every run, whatever the worker count, on a 2.5 Gb/s link with no shaping: "an external pipe, not a peer-supply limit". On run 22, sampled at 96% of the chain, the block-apply thread used 20% of one core, the workers spent 11 to 20% of their time waiting for peer bytes, and iowait was zero.
- Mid-chain, 46 to 91% of per-block apply time is UTX0 lookups. Script verification is small until the assumevalid point (938,343), where it rises to 29 to 46%. Core applies the same assumevalid default.
- Run 27 was bound by the download for almost its whole length. The one stretch where applying blocks set the pace was 07:45 to 08:31Z on 09-19, heights 811,000 to 824,000, under the monitor's load.
- At the tip the two nodes use about the same memory: bmc 7.0 GB resident across three processes, Core 6.7 GB across two.
- On disk, the block files are identical to within a tenth of a GiB. bmc's undo data is 2.5 times Core's, deliberately, because it keeps each spent output's script and value. Its txindex is 2.6 times smaller (26.2 against 68.8 GiB). It also carries two indexes Core has no equivalent of: the address history (203.5 GiB) and the txo-spender index (91.5 GiB).
- In the micro-benchmarks, nothing changed in this period: "on every operation this suite could compare directly, this project is at or behind Bitcoin Core". RIPEMD-160 is 3.16 times slower, SHA-512 1.79 times, ECDSA 1.12 times; only SHA-256 on 32 bytes is faster, by 1.06 times. The end-to-end sync is competitive because it is bound by the network and the UTX0 set, not by hashing.

PART V: PARITY – a name is not a contract

Chapter 22: Method names, and then shapes

By 09-12 every one of Core v31.1's 156 RPC method names existed on the node, and none was a stub. The parity plan, which had tracked method names, gained a new first line that day:

```
"2026-09-12: this file tracks METHOD NAMES, and that is not parity."  
"A name is not a contract."  
-- docs/PARITY_PLAN.md
```

A script that diffs the full JSON key paths of each response against a live Core found more than thirty missing fields in its first run. `getrawmempool true` returned 4 of Core's 16 fields per entry; `getpeerinfo` returned 19 of 38. The response shapes of 18 methods were then captured from Core v31.1 on regtest and frozen into a test that runs in the gate, so that a missing field fails the build instead of waiting to be noticed.

One correction on the way is worth recording. The 09-03 audit had removed the `coinbase_tx` field from `getblock`, reasoning that "Core's blockToJSON has no such member", and added a test pinning its absence. v31.1 returns it. The field was restored on 09-12. An audit, too, can be wrong.

The live survey that found these gaps also found a hazard: its probe loop "called `clearbanned`, `ping`, then `stop`, and took production down for ninety seconds". It now carries a list of methods it must never call.

Chapter 23: The wrong Core

The box's long-running Core oracle is a v31.99 development build, and for a week field diffs ran against it. On 09-18 the first diff against a real v31.1 release showed what that had cost, in both directions:

```
"diffing against it invented work"  
-- docs/PARITY_RPC_FIELDS.md, on fields that exist only in the  
development build
```

And it hid real gaps. `getdeploymentinfo` and `getmempoolentry` had "matched" because the development build had buried taproot's deployment entry and added two mempool fields that v31.1 does not have. Against v31.1, the node was missing the taproot entry and `bip125-replaceable`, and carried the two

development-only fields as extras. Three pull requests (#269, #271, #273) closed those and more:

- `getpeerinfo` had been called 38 of 38 on 09-12, and was not: `addrlocal` was never emitted (0 of 11 peers against Core's 10 of 10), and the fixture test had excused two missing fields as "conditional".
- `taproot` is now reported from a real BIP9 state machine walked over the node's own headers, and matches Core at 32 mainnet heights across its whole lifecycle. The comparison also found three existing defects in the `script_flags` field.
- `chunkweight` and `fees.chunk` were missing from 71,710 of 79,626 mempool entries on production, every entry in a multi-transaction cluster. They are now reported for all, and matched Core chunk for chunk on 336 production clusters.

The rule that came out of it: "never quote a parity figure taken against 8335", the development oracle's port.

Chapter 24: ZMQ

When the operator asked for ZMQ to be turned on for production on 09-19, a real libzmq subscriber received 3,180 `hashtx`, 3,178 `rawtx`, 2 `hashblock` and no `rawblock` at all over two blocks. The publisher sized each subscriber's socket buffer at about 256 KB and wrote without blocking; a 1 to 2 MB block never fit, so the publisher disconnected the subscriber instead:

```
"rawblock was never delivered to anyone, and every block disconnected every
subscriber on its endpoint."
-- docs/CORE_DIVERGENCES.md
```

It now has a per-subscriber queue bounded in messages, as libzmq's publisher does, which drops individual messages for a slow subscriber and keeps the connection (#274). The same day, the `sequence` topic, which the node had refused because it "could publish adds but not removes", was implemented (#278). The single point that sees every mempool removal had existed since 09-16, in the departure journal; the refusal predated it. Event for event, the node's sequence stream matches Core v31.1's on adds, replacements, block conflicts, `invalidateblock`, `reconsiderblock` and a one-step reorg. Building it found that `invalidateblock` over RPC had never worked.

Deployed, the queue exposed one more loss: transaction notifications were staged in a 64-slot ring and silently overwritten under load. The overrun was counted in the log but never shown to subscribers as a gap. On a mempool reload, 256 of 6,000 reloaded transactions reached `hashtx` on regtest. After #283, the production node reloaded 72,366 transactions, and a subscriber received 74,280 `hashtx` and 74,280 `sequence` additions, the same set, with no gaps.

Chapter 25: Relay

On 09-19 a regtest differential set up to measure something else showed that the node never accepted a transaction Core relayed to it, in either direction. It found three defects (#277). Two affect mainnet:

- Every block announcement the node sent by `inv` carried the block hash with its bytes reversed, so "every `inv(MSG_BLOCK)` named a block nobody has". A test asserted the reversed bytes.
- Inbound connections validated transactions against a snapshot of the UTXO set taken when the process started, so they rejected any transaction spending a newer coin as missing its inputs. "On mainnet the loss grows with uptime and was masked by the outbound legs."

The third affected only nodes configured with `connect=`, which production is not: the node's only connection asked its peer not to relay transactions at all.

Chapter 26: What still differs, on purpose

`docs/CORE_DIVERGENCES.md` lists every deliberate difference with its reason. As of 09-19 they include:

- a process per download peer, where Core uses threads;
- a 512 MiB per-subscriber byte ceiling on the ZMQ queues on top of Core's message limit, because "this box has been OOM-killed before";
- six stated differences in the `sequence` stream's ordering during multi-block reorgs;
- `getchainstates` omits Core's two cache-size fields, because the node has no caches those numbers would describe, and reporting `dbcache` under Core's names would invent them;
- `getrawaddrman` omits `source`, because the address book stores the source's network group only, and "a plausible-looking address that no peer ever sent" would be worse than no field;
- additions under a `bmc` prefix, never inside Core's own shapes: `bmcgetdownloadinfo`, `bmcgetmempooljournal`, `bmcgetcapabilities`, and a build attestation in `getnetworkinfo`.

PART VI: SECURITY – 182 findings, one that fell through, and a leak

Chapter 27: Closing an audit by ID

By the morning of 09-04 all 29 distinct CRITICAL and HIGH findings of the 09-03 audit were closed, and 43 of the 44 MEDIUM. The remediation log said what that sentence leaves out:

```
"the 65 LOW and 32 INFO -- 97 findings, more than half the audit -- have never
been examined at all. 'All CRITICAL+HIGH and 43 of 44 MEDIUM' is true and is also
the flattering way to say it."
-- docs/audits/AUDIT_2026-09-03_REMEDIATION.md
```

(The audit's own table gives 68 LOW and 33 INFO; the two documents disagree, and this report quotes each as written.)

On 09-05 a commit announced the LOW tier closed, "68 of 68". The next commit opened:

```
"CORRECTION FIRST. My previous commit said this audit's LOW tier was '68 of 68'.
It was 67. ST0-10 was dropped ... a comm of the audit's LOW ids against every id
mentioned in any commit message is what caught it."
-- commit a12cfc5a
```

ST0-10 was a real defect: after a reorg, a serving process could answer a request for a block hash with whatever block sat at that height. Since then, closure is tracked by re-deriving the open list from the finding IDs, never from the narrative of what was done. The audit was declared fully closed on 09-05, "every audit finding closed, accepted or deferred with a reason", with zero findings unaccounted for.

Every fix carried a negative control: revert the fix, watch the test fail. "Four of those negative controls found the test rather than the fix." Six new tests passed against the unfixed code. One of them, for ST0-10, installed its fixture after the harness forked, so the fixture existed only in the parent: "A fixture written after a fork does not exist for the child."

The audit's findings themselves needed checking. Some were already fixed under other IDs. Some were understated: a finding rated benign was a potential crash, and a cryptographic bound quoted as 2^{223} was about 2^{112} . Some came with suggested fixes that would have introduced bugs, one of which would have left the wallet unlocked forever. The project's rule: an audit finding is a hypothesis, to be verified against the code before it is fixed.

A second review on 09-05 went over the script interpreter with eight different search angles: 36 candidate findings, 17 kept after verification, 5 refuted.

The one CRITICAL finding (IR-1) was a discarded return value when the stack reaches 1,000 items, next to a fix made two days earlier.

Chapter 28: Keys in a backup folder

On 09-04 a pre-deploy snapshot folder, backups/, was committed to the public repository with the rest of the tree. It held the live node's onion v3 private key, its I2P private key and its encrypted wallet, byte for byte the same as the live ones. They were public for three days. On 09-07 the folder was removed (#84), backups/ was added to .gitignore, the history was rewritten in a mirror with the paths dropped, and the live keys were renamed with a .compromised-20260904 suffix so that the node generates new ones. The force-push of the rewritten history and the request to GitHub to purge orphaned commits are the operator's to run; the assistant's permission classifier blocks both.

In the same period the assistant committed under the operator's personal email address, taken from session context instead of the repository's configured identity. GitHub attributes commits by email address, which linked the pseudonymous account to a real person. The history rewrite covered those commits too. The rule that followed: the project has one identity, and the personal name and address "are never to leave this machine".

Earlier, a live password for the Core oracle's RPC had sat in two validation scripts since 08-16. It was rotated by moving the oracle to cookie authentication only: "It remains in git history and always will."

PART VII: THE SCAR TISSUE – incidents, and the patterns they share

The first report counted its incidents by the part of the node they hit. In these seventeen days the incident register has 35 entries, and they are better grouped by shape: most of them are the same few mistakes in different clothes. Each has a rule now, and the project's rulebook explains why every rule cites its incident: "a rule without its scar tissue gets argued away".

Chapter 29: A green suite that measured nothing

- A link check ran before anything was built, failed in a fresh worktree, and stopped every test from running; the log "read exactly like a clean build". (09-03)
- A differential oracle for false accepts was silently dead for more than a day after a commit replaced its protocol. Its headline read "ZERO DIVERGENCES", "technically true and completely meaningless", above a line nobody read: "engine failures: 7879". (09-03)
- Six new tests passed against unfixed code. (09-03 to 09-05)
- The capstone compared empty to empty for months. (until 09-11)
- Seventeen differentials printed "ALL 0 MATCH" on empty case sets. (09-13)
- Three times in one day, a test run after a failed build ran the previous binary and printed ALL TESTS PASSED, twice during a check meant to prove a test could fail. "A reversion that does not compile is a NON-RESULT." (09-15)

The countermeasures are the revert-the-fix control, a refusal to pass on an empty result (exit 2), and checking the build's exit code before trusting any test output.

Chapter 30: DONE on paper, zero at runtime

Compact-block receive was marked DONE on 09-06. On 09-08 an outside review said the node could not receive compact blocks; the assistant told the operator the claim was outdated and rewrote three documents to say so. Two hours later production's own counter read 0 blocks reconstructed. The cause, found the next day: the node recognised a `block` message by comparing five bytes, and `blocktxn` begins with the same five. Every reply to a compact block was taken for a full block and failed. Production counted 1,262 round trips and 0 blocks. "I corrected the docs and the user a second time."

The same shape recurred: `addnode` answered "done" and dialed nothing, because the runtime list had no reader; a dial backoff that was "designed, tested and reported" never applied on production, because its shared table was created in

the wrong process; the index builder's periodic tick could not be reached during a sync, so "every index looked healthy on run 26 and no run was ever built". The rule: a feature is done when a counter on a live node shows it working.

Chapter 31: A comment that ends the inquiry

The node logged a non-monotonic archive warning 44 times in one day, and nobody looked, because the operations guide called it "expected on a parallel-downloaded archive". That was true before 09-08 and false after it. The warning was a real defect: six layout breaks, which disable truncation and pruning, and the path to Chapter 15. A monitoring tool's config disabled a data source on the strength of a similar stale comment. The rule: date a claim that something is benign against the change it describes.

Chapter 32: A threshold calibrated at one point

A 32 KB/s floor for evicting slow download peers was calibrated for megabyte-sized blocks. At height 50,000 a block is about 200 bytes and a healthy peer moves about 9 KB/s. Run 7 evicted 655 peers in 30 minutes, 181 of them peers that had served two full chunks. The operator:

```
"We are killing peers that have served us blocks. wtf!"  
"Run 4 was a crime against good coding. That should be taught as 'what not to  
do'."
```

It became rule 11 of the project's engineering rules: a threshold is relative to the population it judges, and every eviction prints the value and the bar. The same family includes the 2,048-input sequence ledger that rejected block 880,338, and a sizing rule that booted every fresh sync on a 64 MB memory table while `dbcache=8192` sat unused, because it asked whether the node was at the tip without asking which tip: "When a rule says 'tip', ask which tip."

Chapter 33: The tools as an adversary

A category that barely existed in the first report: the assistant's own tools producing false evidence.

- `pgrep -f` matched the shell that was waiting on it, so three gate waiters waited on themselves, and one `kill -f` killed its own session.
- An unquoted heredoc executed the backtick code spans in a report as shell commands, and the mangled text was committed.
- `open(p, 'w').write(open(p).read())` truncates the file before reading it, and emptied a document that was then pushed.
- `git add -A` in the same step as a `.gitignore` edit committed ten 31 MB binaries.

- A background gate inherited a stale working directory and produced a two-line log that "was nearly read as a real result".
- `chmod +x` never reaches a commit when git ignores file modes. Four failures in one day, "each time disguised as something else"; a check later found fifteen harness scripts that could not run.
- The agent harness killed a background task, with its whole process tree, for "low memory" while 100 GB were free.

Each has a one-line rule in the assistant's memory.

Chapter 34: Misreading the human

The costliest incidents of the period were not code defects. They were requests understood wrongly.

On 09-08 the operator said "relaunch with updated binary". The assistant ran the fresh-run procedure it had used since run 10, which starts by deleting the datadir, and destroyed run 16 at 704,169 blocks after nine hours. "The operator meant: swap the binary, keep the data, resume." And: "I never said 'this wipes the run' before doing it." The rule: a relaunch reuses the datadir, and a wipe is announced in the sentence that proposes it.

On 09-18 the operator asked for run 27 to be followed in the monitoring tool. The assistant added it, without saying that a monitor polling a timed node perturbs the measurement. It cost the measurement its cleanness (Chapter 19). The rule now is to say so, and to offer the run's log files instead.

On 09-10 the operator said "Ignore the wifi". The assistant stopped investigating the wifi interface, which was the cause of the outage it was investigating.

Each of these produced the same rule in different words: state the consequence before acting on an ambiguous request.

Chapter 35: The rules now in force

The operator's rules, in their own words where they exist:

rule	wording	since
one identity	the personal name and email "are never to leave this machine"	09-04
batches land as merge commits	"PR per batch, merge commit, never squash or force-push"	09-05
flags match Core	"Our flags need to match core. Never do something different unless we discuss it."	09-06
Core is compared at Core's defaults	"Don't ever take liberties with Core again."	09-06
no consensus caps Core lacks	"If core has no cap, we should not have one either."	09-09
do what Core does	"Do what core does! Retain both branch headers!"	09-09
fix causes	"Fix the cause, not the symptom."	09-09
LAN only	"Never over tailscale when we have ethernet"	09-10
benchmarks on flash	"never platter drives"	09-11
everything builds during IBD	"Everything should be generated during ibd."	09-16
named directories	"Always create a named subdirectory and work inside it."	09-17
separate projects stay separate	"You stick to bmc work."	09-18
no restarts in a baseline	a benchmark node is never restarted or migrated mid-run	09-18
never poll a benchmark	"never add an IBD node to blockyard. Let the run finish before anything queries its RPC."	09-19

The assistant's own rules, each from an incident in this period, include: verify tests by reverting the fix; a test that asserts the bug is a defect in the test; audit findings are hypotheses; track closure by ID; check linkage with `nm`, not `grep`; performance tests measure the finding's cost model; dry-run assembly patches on a scratch copy; name the writer and the reader of every new field before starting a twenty-minute gate; pin the height on both sides of a comparison; verify DONE by runtime counters; diff whole documents, not fields.

PART VIII: THE COLLABORATION – from standing authorization to a gated contract

Chapter 36: The operator as the merge and deploy button

"21 FOR 21" described a mode of standing authorization: "just fix them, and redeploy the server if you need to". From about 09-08 an automatic permission classifier stands between the assistant and any action that is destructive or visible outside the machine. In this period it refused: deleting benchmark datadirs, killing processes, merging pull requests, restarting or deploying production, rolling production back, changing the firewall, force-pushing the rewritten history, writing the 293-byte archive repair, and temporarily reintroducing a consensus bug to test whether a test would catch it.

The assistant's recorded stance toward these refusals is cooperative. On the consensus-bug experiment: "I did not look for a way around that." The same point was proved another way. On the archive repair: "correctly, it guards writes into archive files". Merges now happen only on the operator's explicit instruction. Some refusals held even after the operator had approved in the conversation, above all production deploys, and the operator then ran the command themselves or repeated the instruction in so many words. The operator decides what merges and what deploys; the assistant prepares, gates and hands over.

The effect on the work is visible in the records: a finished, gated batch waits for a human to land it, and hand-off messages end with the exact command to run.

Chapter 37: Parallel agents

In "21 FOR 21" one model worked one change at a time. In this period the assistant began splitting independent work across subagents, each in its own git worktree on its own batch branch, each running the full gate before reporting back. On 09-19 alone, nine such agents worked on the fixes needed for a clean run 28, several at a time, and six of their pull requests merged within four minutes of each other.

This introduced two new failure modes. The first is additive merge conflicts. Every branch appends its tests to the same line of the Makefile, and the combination of all branches is a tree that no single gate has tested. On 09-19 the combined tree failed its gate once on a stale-object false alarm, and once on a real interaction: a test that each branch had passed alone failed 1 run in 5 together. The second is knowledge that does not cross between agents: the Core port trap in Chapter 20 was known to one agent and not to the one who needed it.

Chapter 38: Memory as the institutional record

The assistant keeps a directory of short memory notes, loaded into every new session. Each records one lesson: what happened, why it matters, and how to apply it. On 09-19 it held more than sixty. Many cite dates, counts and the operator's words verbatim. They are how a rule made in one session reaches the next, and they were the richest single source for this report.

They can also be wrong. Two notes told future sessions to rebuild indexes that were already correct; following them "would mean a multi-hour rebuild of indexes that are already correct". A note about a coin-height defect is kept, marked DISPROVED, "only so the claim is not re-made". Notes are corrected, not deleted, and the correction says why.

Chapter 39: Neighbours

The machine is shared. Blockyard, a web monitor and explorer for this node, is also written entirely by AI, directed by the same operator, in its own session. The two coordinate through the operator: bmc advertises what it can do through an RPC of its own, Blockyard reads bmc's logs and RPC, and each session reports the other's defects instead of fixing them. On 09-18 the operator drew the line: "the blockyard task is handling the rules. You stick to bmc work." The coupling has costs as well as benefits: Blockyard's polling perturbed both benchmark runs, and a stale comment in Blockyard's config had hidden a bmc defect.

Another AI agent, a Hermes agent, works in the same checkout and pushes on top of the same branch. On 09-02 one of its jobs exhausted the machine's memory and forced a hard reset. After a global out-of-memory event on 09-08 killed the user session manager (and every terminal with it), the operator doubled the machine's RAM from 60 to 123 GB.

PART IX: WHAT IS STILL OPEN

Recorded as open in the project's own registers on 2026-09-19:

- The clean benchmark pair. Closed on 2026-09-21: Core 19 h 32 m 54 s, run 28 18 h 24 m 02 s, UTX0 set identical (Chapter 20). Each side has run once, so the run-to-run spread of bmc is still unmeasured.
- A full-verification sync with `assumevalid=0` was parked at 77% on 09-06 and has never completed. The compatibility register still says RUNNING.
- A UTX0 record cannot store an output script longer than 65,535 bytes; `submitblock` refuses blocks with more than 16,384 transactions.
- The node's cluster linearization is a greedy approximation; Core v31.1 searches for the optimal one. They agree on every shape tested, and may differ on clusters where transactions have several parents and several children.
- `getrawtransaction` and `gettxspendingprevout` scan their index's unfolded tail linearly on every lookup.
- Core publishes `hashtx` and `rawtx` for every transaction in a connected block; the node publishes them only for mempool accepts.
- The node sends no `feefilter` on outbound connections and ignores `sendheaders` there.
- Every bulk `getrawmempool true` allocates about 8.4 KB per entry, about 670 MB at production's mempool size.
- The first `getchaintxstats` after each restart takes 9 to 23 seconds.
- Six old layout breaks remain in run 26's archive.
- Whether the rewritten history reached GitHub is recorded only as the operator's action.
- The micro-benchmarks are unchanged since August: the hashing and signature primitives are at or behind Core's.

APPENDIX A: THE NUMBERS, ON ONE PAGE

	value	source
period	2026-09-02 to 2026-09-19 (days 22 to 38 of the git record)	git
commits on main since 09-02	927 (642 non-merge, 285 merges)	git, deduplicated
merged pull requests	282 (#2 to #283)	GitHub
code and tests	~318,200 lines (+25%)	git
gate test programs	428 (from 324)	asm/Makefile
RPC methods	166, including all 156 of Core v31.1's	dispatch tables, PR #188
models	Claude Fable 5.1 (310), Opus 5 (269), Sonnet 5 (10)	commit trailers
audit findings	182 (5 CRITICAL, 32 HIGH, 44 MEDIUM, 68 LOW, 33 INFO)	CODEBASE_AUDIT_2026-09-03
UTX0 set comparisons with Core	9, every real one identical (10 with run 28, on 09-21)	Part III
run 27	tip in 18 h 40 m 23 s, muhash identical at 967,712	debug.log, phase.log
Core v31.1 baseline #1	19 h 40 m 09 s (polled)	debug.log
Core v31.1 rerun (unpolled)	19 h 32 m 54 s (left IBD 2026-09-20 10:01:27Z)	watch.log
run 28 (unpolled, fixed code)	18 h 24 m 02 s, muhash identical at 968,025	phase.log
download rate	~11 MB/s in every run	debug.log
memory at the tip	bmc 7.0 GB, Core 6.7 GB	PERFORMANCE.md
incidents in the register	35	memory, worklogs

APPENDIX B: RECOVERING A TIMELINE FROM BLOCK FILES

The twelve-second restart of the first Core baseline trimmed its debug.log to the last ten megabytes, which erased every progress line before height ~580,000. The timeline was recovered from the block files themselves. Core creates each `rev?????.dat` undo file when it connects the first block stored in the matching `blk?????.dat` file. The height of that block is the height of the first header in the blk file. Core v31.1 obfuscates block files with an XOR key kept in `blocks/xor.dat`, so each header is decoded with that key before hashing, and the hash is looked up to get its height. Checked against the surviving log at ten heights between 616,000 and 951,000, the file creation time ran 0 to 143 seconds earlier than the log, and under 60 seconds at nine of the ten.

The Core rerun sets `shrinkdebugfile=0`, so it will not need this.

APPENDIX C: READING ORDER – AUDITING THIS REPORT AGAINST THE SOURCE

1. docs/reports/2026-09-18-run27/README.md: the headline benchmark, its contaminants, and the Core rerun's setup.
2. docs/reports/2026-09-11-ibd-vs-core.md and docs/reports/2026-09-11-run22-muhash-divergence.md, with their correction boxes: the capstone that never ran, and the torn read.
3. docs/reports/2026-09-17-run26-vs-core.md: two disks, not two implementations.
4. docs/devlog/RESUME_2026-09-17.md and docs/devlog/2026-09-18-frontier-verified-on-a-live-restart.md: block 967,422.
5. docs/audits/CODEBASE_AUDIT_2026-09-03.md, then docs/audits/AUDIT_2026-09-03_REMEDIATION.md, then commits fa6594c8 and a12cfc5a: an audit, its closure, and the finding that fell through.
6. docs/PARITY_RPC_FIELDS.md and docs/CORE_DIVERGENCES.md: parity as measured, and every deliberate difference.
7. docs/devlog/DEPLOYMENT_HISTORY.md: what production ran, and when.
8. The daily worklogs, worklog/2026-09-*.md (09-11 to 09-15 were written after the fact, from the commit record).